Mini Shell – Technical Report

Submitted BY: Chirag Suthar (2025MCS2105), RajKumar (2025MCS2744)

Course: COD 7001 (System Concepts)

1 Introduction

This technical report explains the internal design, algorithms, and system programming concepts used to build the Mini Shell. The project provides a simplified but functionally meaningful model of a UNIX command shell, demonstrating:

- Tokenization and parsing
- Execution of programs using `fork()` and `execvp()`
- Input/output redirection (single)
- Pipeline execution using pipes (single)
- Background process handling
- Signal handling (`SIGINT`, `SIGCHLD`)
- Memory management and error recovery

The Mini Shell is implemented in modular C source files:

- `mini-shell.c` – main loop, prompt, built-ins, signal handling
- `include/tokenizer.c` – lexical analyzer
- `include/parser.c` – syntax analyzer, command struct builder
- `include/execute.c` – execution engine
- `include/history.c` – maintain history functions

This structure closely follows the architecture of real shells such as `bash`, though in a simplified form.

2 System Architecture Overview

The Mini Shell follows this pipeline:

$$UserInput \rightarrow Tokenizer \rightarrow Parser \rightarrow CommandStructure \rightarrow Executor \rightarrow Terminal$$

2.1 Tokenizer

The tokenizer converts raw input into meaningful tokens. It handles:

- Words
- Operators: `<`, `>`, `>>`, `|`, `&`
- Quoted strings (“ . . . ” or ‘ . . . ’)
- Whitespace compression

If anything unsupported appears, it raises an error. Quoted strings remain intact as single tokens.

2.2 Command Parsing

The parser constructs the internal structure `command_t`.

2.2.1 Responsibilities of the Parser

1. Validates token order and syntax
2. Identifies input/output redirection
3. Detects background operator (`&`)
4. Identifies single pipeline operator (`|`)
5. Builds `argv[]` dynamically
6. Fills the structure fields: `argv`, `infile`, `outfile`, `append`, `background`

2.2.2 Error Handling in Parsing

The parser detects:

- Missing filenames after redirection operators
- Multiple redirections
- Misplaced `&`
- Pipeline at start or end
- Multiple pipeline operators (unsupported)
- Unmatched quotes

2.3 Command Execution Engine

2.3.1 Execution of Single Commands

1. Shell creates a child using `fork()`
2. Child process:
   - Restores default `SIGINT` behavior
   - Sets up redirection with `open()` and `dup2()`
   - Calls `execvp()`
3. Parent process:
   - Waits for child (foreground)
   - Does not wait (background)

2.3.2 Redirection Handling

- `<` uses `O_RDONLY`
- `>` uses `O_WRONLY | O_CREAT | O_TRUNC`
- `>>` uses `O_APPEND`

The redirection uses:

```
dup2(fd, target_fd)
```

2.4 Pipeline Execution

Pipelines allow:

```
cmd1 | cmd2
```

Execution steps:

1. Create pipe with `pipe(pipefd)`
2. Fork left command, redirect stdout to write-end
3. Fork right command, redirect stdin from read-end
4. Parent closes pipe ends
5. Parent waits for both children

Limitations:

- Only one pipeline supported

3 Signal Handling

3.1 SIGINT (Ctrl+C)

- Shell should not terminate
- Only child should be interrupted
- Shell installs custom signal handler

3.2 SIGCHLD – Zombie Cleanup

Background processes become zombies. The shell installs:

```
signal(SIGCHLD, handle_zoombie)
```

Inside the handler:

```
waitpid(-1, NULL, WNOHANG)
```

This prevents zombies.

4 Memory Management

Dynamic allocations occur during:

- Token creation
- Argument vector building
- Filename duplication

Cleanup functions prevent memory leaks:

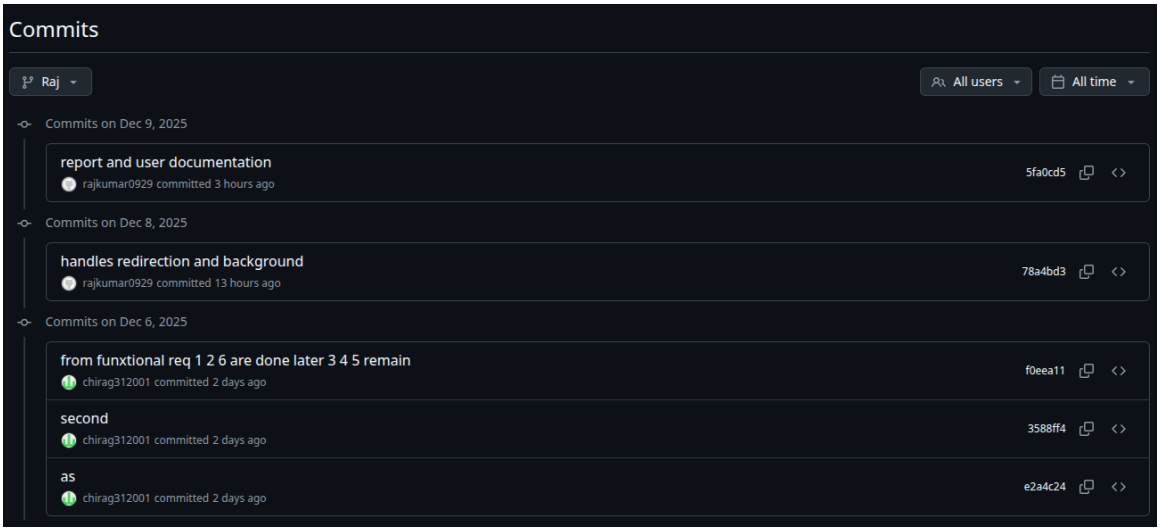
- `free_tokens()`
- `free_memory_cmd()`

5 Design Choices and Simplifications

Feature	Support
Single background operator	Supported
Single pipeline	Supported
Basic redirection	Supported
History feature	Supported
Built-ins (cd, exit)	Supported
Zombie cleanup	Supported
Others...	

6 Git Commit History

RajKumar



Chirag Suthar